

Aula 15 - Implementação da Sprint 1: CI/CD e Testes Automatizados

Duração: 2 horas

Objetivos:

- Implementar pipeline CI/CD com GitHub Actions
- Adicionar testes automatizados para novas funcionalidades
- Garantir qualidade do código com ferramentas automatizadas

Parte Teórica (30 min)

1. Fluxo CI/CD Ideal

Diagram

Code

```
graph LR
    A[Commit] --> B[Build]
    B --> C[Testes Unitários]
    C --> D[Análise SonarQube]
    D --> E{Passou?}
    E -->|Sim| F[Deploy em Staging]
    E -->|Não| G[Notificação de Falha]
```

2. Boas Práticas em Testes

- **Test Pyramid:**
 - 70% Unitários
 - 20% Integração
 - 10% E2E
- **FIRST Principles:**
 - Fast (Rápidos)
 - Isolated (Isolados)
 - Repeatable (Reproduzíveis)
 - Self-validating (Auto-verificáveis)
 - Timely (Escritos no momento adequado)

Atividade Prática (1h30min)

Passo 1: Configurar GitHub Actions

1. Crie `.github/workflows/ci.yml`:

yml

```
name: Java CI Pipeline

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Build and Test
        run: mvn clean verify

      - name: SonarQube Scan
        uses: SonarSource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
          SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }
```

2. Adicione secrets no GitHub:

- SONAR_TOKEN (Gerado no SonarQube)
- SONAR_HOST_URL (URL do seu SonarQube)

Passo 2: Implementar Testes para Nova Funcionalidade

Cenário: Testar o endpoint de busca criado na Aula 14

1. Teste Unitário do Service

java

```
// src/test/java/com/example/projeto/domain/service/ProdutoServiceTest.java
@ExtendWith(MockitoExtension.class)
class ProdutoServiceTest {
    @Mock
    ProdutoRepository repository;

    @InjectMocks
    ProdutoService service;

    @Test
    void deveBuscarProdutosPorNome() {
        when(repository.findByNomeContaining("Xiaomi"))
            .thenReturn(List.of(new Produto("Smartphone Xiaomi")));

        List<Produto> result = service.buscarPorNome("Xiaomi");

        assertEquals(1, result.size());
        verify(repository).findByNomeContaining("Xiaomi");
    }
}
```

2. Teste de Integração com TestContainers

java

```
// src/test/java/com/example/projeto/interface/rest/ProdutoControllerIT.java
@Testcontainers
class ProdutoControllerIT {
    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15");

    @DynamicPropertySource
    static void configure(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
    }

    @Test
    void devePersistirProdutoNoBancoReal() {
        // Teste que usa o PostgreSQL real
    }
}
```

Passo 3: Validar Qualidade

1. Execute localmente:

```
bash
```

```
mvn clean verify sonar:sonar
```

2. Verifique no SonarQube:

- Cobertura de testes
- Bugs críticos
- Code smells

3. Corrija 1 problema identificado (Exemplo comum):

```
java
```

```
// Antes (problema: possível NPE)
public BigDecimal getPrecoFormatado() {
    return preco.setScale(2, RoundingMode.HALF_UP);
}

// Depois (correção)
public BigDecimal getPrecoFormatado() {
    return preco != null ? preco.setScale(2, RoundingMode.HALF_UP) : BigDecimal.ZERO;
}
```

Checklist de Entrega

- Pipeline CI funcionando no GitHub
- 2 novos testes implementados
- 1 melhoria aplicada baseada no SonarQube
- PR aberto com label `sprint-1`

Tarefa Pós-Aula

1. Adicione um estágio de deploy para Heroku no CI
2. Escreva um teste para validação de campos obrigatórios

Exemplo de deploy step:

```
yaml
```

```
- name: Deploy to Heroku
  if: github.ref == 'refs/heads/main'
  run: |
    curl https://cli-assets.heroku.com/install.sh | sh
    heroku deploy:jar target/*.jar --app ${ secrets.HEROKU_APP_NAME }
```

Próximos Passos

- **Aula 16:** Refatoração e Design Patterns
- **Aula 17:** Integração com Frontend

Dica do Expert:

"Use `@DataJpaTest` para testar apenas a camada JPA isoladamente!"

Recursos:

- [GitHub Actions para Java](#)
- [TestContainers Guide](#)

Pronto para elevar seu projeto a um nível profissional? 🚀 🔥